DSA - Microsoft

Two Sum

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int,int> mp;

        for (int i = 0; i<nums.size(); i++){
            int complement = target - nums[i];

            if (mp.count(complement))
                return {mp[complement], i};

            mp[nums[i]] = i;
        }

        return {};
    }
};
```

Longest Substring without repeating characters

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_map<char,int> mp;
        int left = 0;
        int maxLen = 0;

        for (int right = 0; right<s.size(); right++){
            if (mp.find(s[right]) != mp.end())
            left = max(left, mp[s[right]]+1);

            mp[s[right]] = right;
            maxLen = max(maxLen, right - left +1);
        }

        return maxLen;
    }
};
```

Merge Sorted Array

```
class Solution {
public:
```

```
    void merge(vector<int>& nums1, int m, vector<int>&
nums2, int n) {
        int i = m-1;
        int j = n-1;
        int k = m+n-1;

        while(j>=0){
            if (i>=0 && nums1[i] > nums2[j]){
                nums1[k--] = nums1[i--];
            }else{
                nums1[k--] = nums2[j--];
            }
        }
    }
};
```

Maximum Subarray

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int n = nums.size();
        vector<int> dp(n);//create dp array

        dp[0]=nums[0];
        int ans=dp[0];//create ans integer

        for (int i=1;i<n;i++){
            dp[i] = max(nums[i],dp[i-1]+nums[i]);
            //max sum subaarray
            ans = max(ans,dp[i]);
        }

        return ans;
    }
};
```

Climbing stairs

```
class Solution {
public:
    int climbStairs(int n) {
        if (n<=2)
            return n;

        vector<int> dp(n+1);
```

```
        dp[1] = 1;
        dp[2] = 2;

        for (int i=3; i<=n; i++){
            dp[i] = dp[i-1] + dp[i-2];
        }

        return dp[n];
    }
};
```

Number of islands

```
class Solution {
public:
    void dfs(int i, int j, vector<vector<char>> &grid){
        if (i<0 || i>=grid.size() || j<0 ||
j>=grid[0].size() || grid[i][j] == '0')
        return;

        grid[i][j] = '0';//cell is visited

        dfs(i+1,j,grid);
        dfs(i-1,j,grid);
        dfs(i,j+1,grid);
        dfs(i,j-1,grid);
    }

    int numIslands(vector<vector<char>>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        int count = 0;

        for (int i=0; i<m; i++){
            for (int j=0; j<n; j++){
                if (grid[i][j] == '1'){
                count++;
                dfs(i,j,grid);
                }
            }
        }

        return count;
    }
};
```

Sub-Array Sum equals k

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int,int> mp;
        mp[0] = 1;

        int prefixSum = 0;
        int count = 0;

        for (int num:nums){
            prefixSum += num;
            if (mp.count(prefixSum-k))
            count += mp[prefixSum-k];

            mp[prefixSum]++;
        }

        return count;
    }
};
```

Best time to buy and sell stock

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minPrice = INT_MAX;
        int maxProfit = 0;

        for (int price:prices){
            minPrice = min(minPrice, price);
            maxProfit = max(maxProfit, price - minPrice);
        }

        return maxProfit;
    }
};
```